# An?lisis del incidente P1-8841: hechos, hip? tesis y conclusiones

**Servicio:** OrionOrders **Fecha del incidente:** 29 de septiembre de 2026 **Ventana con evidencia:** 09:21–10:41:30 **Estado:** análisis documental; la recuperación y la integridad de pedidos no están verificadas.

## 1. Propósito y alcance

Este informe separa **hechos comprobables**, **hipótesis causales** y **conclusiones derivadas** para evitar que una coincidencia temporal o una declaración operativa se presente como causa raíz confirmada. Correlaciona publicación, aplicación, pool HTTP, infraestructura Kubernetes, balanceador, servicio `customer-profile` y base de datos.

El análisis utiliza únicamente los artefactos suministrados en `ORIONORDERS/`. No afirma que se haya ejecutado una mitigación, que el incidente haya terminado ni que todos los pedidos estén íntegros. No autoriza acciones en producción.

### Escala de certeza

- **Alta:** observación repetida directamente en una fuente primaria.
- **Moderada-alta:** varias fuentes independientes son consistentes, aunque

faltan controles para aislar causalidad.

- **Moderada:** mecanismo plausible con evidencia parcial y explicaciones

alternativas todavía abiertas.

- **Baja / no demostrada:** evidencia insuficiente para sostener la afirmación.

## 2. Resumen ejecutivo

La evidencia primaria muestra una saturación del pool HTTP cliente de `orders-api` hacia `customer-profile`: a las 09:43:20 se registran `leased=20 max=20 pending=19`; la cola y `clientPoolWaitMs` aumentan después, mientras los logs de las mismas solicitudes registran tiempos downstream mucho menores. La release 4.18.0 documenta que el máximo de conexiones pasó de 200 a 20 poco antes del inicio del incidente.

La reducción a 20 es la **hipótesis principal** para explicar la saturación, pero no se puede declarar causa raíz confirmada: no se proporcionó el historial de configuración efectiva por instancia ni una prueba controlada que aísle el cambio. Los fallos de liveness y la pérdida de pods comienzan después de la primera espera y pueden haber amplificado el impacto.

La alarma de base de datos corresponde a `db-reporting-01`, mientras que `orders-api` usa `orders-primary-01`. En la comprobación de las 10:02, la base primaria reporta 118/300 conexiones, CPU 44% y cero esperas por bloqueo; esta instantánea no respalda saturación de la primaria en ese momento.

La última muestra agregada del balanceador es de las 10:15. El log de `orders-api` continúa hasta las 10:41:30 y aún muestra 9000 ms de espera y 128 solicitudes pendientes. No hay telemetría posterior que demuestre recuperación, ni reconciliación que permita confirmar la integridad completa de los pedidos.

## 3. Hechos comprobados

| ID | Hecho | Evidencia directa | Alcance y límite |
|---|---|---|---|
| F-01 | 4.18.0 se desplegó; rollout completo reportado a las 09:30. El pipeline registra build, 486 pruebas unitarias, análisis estático, escaneo de contenedor, Helm, deploy, rollout y smoke tests como PASS. | `ORIONORDERS/05_DEVOPS_CICD_RELEASE/04_pipeline_4.18.0(3).log`; `ORIONORDERS/01_SOPORTE_INCIDENTE/02_P1-8841(2).txt`. | El pipeline indica que no había validación de rango para el máximo del pool ni prueba de carga similar a producción. |
| F-02 | La nota de release registra una reducción de `CUSTOMER_PROFILE_MAX_CONNECTIONS` de 200 a 20. Los timeouts de conexión/lectura se mantienen en 1000/2000 ms. | `ORIONORDERS/05_DEVOPS_CICD_RELEASE/03_RELEASE_4.18.0(2).md`. | Es el valor documentado en la release; falta evidencia de la configuración realmente cargada por cada pod. |
| F-03 | La ficha del incidente sitúa la primera alerta de latencia a las 09:42 y la declaración P1 a las 09:49. Reporta cuatro clientes empresariales afectados, operaciones intermitentes, p95 > 8 s y 502/503 entre 12% y 35%. | `ORIONORDERS/01_SOPORTE_INCIDENTE/02_P1-8841(2).txt`. | Impacto resumido por la ficha; no proporciona conteo conciliado de pedidos fallidos/perdidos. |
| F-04 | El balanceador muestra p95 0.7 s y 5xx 0.4% a las 09:41; a las 09:42 muestra p95 1.48 s y 5xx 4.94%; a las 09:44 muestra p95 2.17 s y 5xx 6.57%. | `ORIONORDERS/03_INFRAESTRUCTURA/10_load_balancer_metrics(2).csv`. | Métricas agregadas por minuto; no equivalen a errores o pedidos por cliente individual. |
| F-05 | El log de aplicación registra un aumento de `clientPoolWaitMs` desde las 09:42. A las 09:43:20 observa `leased=20 max=20 pending=19`; a las 09:44:10 registra espera 1778 ms y 25 pendientes. | `ORIONORDERS/02_APLICACION_DESARROLLO/05_orders-api(2).log`. | Observaciones directas de pool y solicitudes muestreadas; no son percentiles agregados. |
| F-06 | A las 09:53:10 una solicitud 502 registra 3549 ms de espera de pool y 118 ms de duración customer-profile; a las 09:54:10, otra solicitud 502 registra 4435 ms y 89 ms, respectivamente. | Log de `orders-api`. | Comparación dentro de solicitudes individuales; no descarta demoras no capturadas en otros requests. |
| F-07 | Customer-profile agregado reporta p95 de 87–143 ms y errores menores de 0.4% durante el periodo disponible 09:30–10:15. | `ORIONORDERS/02_APLICACION_DESARROLLO/08_customer_profile_metrics(2).csv`. | Agregado del servicio dependiente y limitado al periodo del archivo; no demuestra que toda llamada estuviera sana. |

| ID | Hecho | Evidencia directa | Alcance y límite |
|---|---|---|---|
| F-08 | APM registra crecimiento de `customer_profile_pool_wait_ms` desde 09:42, con muestras de cientos a miles de milisegundos, mientras `customer_profile_network_ms` aparece mayormente en decenas o pocos cientos de milisegundos. | `ORIONORDERS/02_APLICACION_DESARROLLO/13_apm_trace_breakdown(2).csv`. | Trazas muestreadas cada 20 segundos, no población completa de solicitudes. |
| F-09 | Kubernetes registra fallos de liveness y terminaciones desde 09:45:12; un evento a las 09:54:11 reporta fallo de sonda sin terminación asociada. A las 09:52 HPA fija tamaño deseado 18 por CPU sobre objetivo. | `ORIONORDERS/03_INFRAESTRUCTURA/07_k8s_events(2).log`. | Eventos puntuales. No describir el fallo de 09:54:11 como reinicio confirmado. |
| F-10 | El CSV de pods muestra 12 pods listos al inicio, luego fluctuaciones y 4 listos a las 10:08 y en la última muestra de 10:14. La columna `cumulative_restarts` no es monotónica. | `ORIONORDERS/03_INFRAESTRUCTURA/06_k8s_pod_metrics(2).csv`. | El contador no es fiable como total acumulado; los pods listos son muestras por minuto. |
| F-11 | La alarma de DB señala conexiones 92%/96% y CPU 81% en `db-reporting-01`. El datasource de orders-api apunta a `orders-primary-01`. La comprobación DBA de 10:02 en la primaria registra 118/300 conexiones, CPU 44%, cero lock waiters. | `ORIONORDERS/04_BASE_DE_DATOS/09_db_alerts(2).txt`. | La lectura de la primaria es una instantánea puntual, no una serie de todo el incidente. |
| F-12 | El balanceador tiene su última fila a las 10:15: p95 10.2 s y 5xx 38%. El log de orders-api continúa a las 10:41:30 y termina con status 200, total 9330 ms, espera de 9000 ms y pool 20/20 con 128 pendientes. | CSV del balanceador y log de orders-api. | El status 200 de una muestra no demuestra recuperación; no hay serie agregada del balanceador después de las 10:15. |
| F-13 | No existe en los artefactos revisados un registro de mitigación aplicada, una hora de resolución ni una reconciliación completa de pedidos. | Revisión del conjunto de fuentes enumeradas en `specs/implement/entregables/00-indice-fuentes.md`. | Ausencia en el paquete disponible; no es prueba de que no se hayan realizado acciones por otros medios. |

## 4. Hipótesis evaluadas

| ID | Hipótesis | Evidencia a favor | Evidencia en contra / faltante | Evaluación |
|---|---|---|---|---|
| H-01 | La reducción del máximo de conexiones de 200 a 20 en 4.18.0 causó saturación local del pool y la degradación. | Cambio documentado previo al incidente; 20/20 ocupadas; cola y espera crecen; trazas/logs muestran espera local muy superior al downstream medido. | No se dispone de valor efectivo por pod, historial de rollout/configuración, límites de capacidad downstream ni prueba antes/después bajo carga comparable. | **Hipótesis principal.** Alta confianza en que hubo saturación observada; moderada-alta en que el cambio específico la causó. No confirmada formalmente. |
| H-02 | `customer-profile` estuvo lento y originó la cola. | Es dependencia síncrona; si retiene conexiones, puede aumentar la ocupación del pool. | Métrica agregada y tiempos de red muestreados generalmente ~87–143 ms/decenas a pocos cientos de ms; no explican por sí solos esperas locales de varios segundos. Cobertura incompleta. | Posible contribuyente no descartado para periodos no observados; explicación principal débil con los datos disponibles. |
| H-03 | La base de datos primaria causó la degradación. | Hubo alertas DB durante el incidente. | Las alertas apuntan a `db-reporting-01`, no al datasource `orders-primary-01`; la primaria muestra 118/300 conexiones, 44% CPU y cero bloqueos a las 10:02; APM muestra DB mucho menor que pool wait en muestras. | No respaldada como causa primaria por la evidencia entregada. No se descartan otros periodos no muestreados. |
| H-04 | Los fallos de liveness iniciaron el incidente. | Hubo timeouts y terminaciones de pods. | El síntoma y la espera del pool comienzan a las 09:42; primer timeout de liveness registrado a las 09:45:12. | La secuencia no respalda liveness como iniciador; posible factor de amplificación. |
| H-05 | La pérdida de pods y el HPA agravaron la indisponibilidad. | Eventos de terminación; descenso de pods listos; HPA deseado a 18; chat del equipo indica aumento de errores tras perder pods. | No hay análisis contrafactual ni correlación por pod; contador de reinicios del CSV es anómalo; no se demuestra que pods nuevos estuvieran listos ni aliviasen demanda downstream. | Factor contribuyente plausible, impacto no cuantificado. |

| ID | Hipótesis | Evidencia a favor | Evidencia en contra / faltante | Evaluación |
|---|---|---|---|---|
| H-06 | Las órdenes confirmadas se perdieron o duplicaron durante el incidente. | La ficha indica que pedidos nuevos tardaban o fallaban. | No se suministró conciliación de IDs, estado de confirmación, reintentos o persistencia. La ausencia inicial de evidencia de pérdida no prueba cero pérdidas. | No verificable; no afirmar pérdida ni integridad completa. |

## 5. Conclusiones sustentadas

### C-01 — Hubo saturación del pool HTTP en las muestras

**Conclusión:** a las 09:43:20 el pool estaba en 20 de 20 conexiones ocupadas, con 19 solicitudes pendientes; a las 09:44:10 había 25 pendientes y 1778 ms de espera. Los logs y trazas posteriores muestran que la espera continúa creciendo.

**Certeza: alta** para esas observaciones. Esto demuestra la saturación local registrada, no por sí solo la causa inicial de la saturación.

### C-02 — La demora local explica gran parte de las muestras de solicitud

**Conclusión:** en solicitudes concretas de 09:53–09:54, los tiempos de espera del pool (3549–4435 ms) son muy superiores a la duración downstream registrada (89–120 ms). APM presenta el mismo patrón de forma repetida.

**Certeza: alta** para los registros citados; **moderada-alta** como explicación de la latencia observada de esas solicitudes. Las muestras no sustituyen una distribución completa.

### C-03 — El cambio 200→20 es la explicación causal principal, pero queda como hipótesis

**Conclusión:** el cambio documentado coincide temporalmente con el inicio y es consistente con la cola observada. No puede llamarse causa raíz confirmada sin verificar el valor efectivo por instancia, descartar factores concurrentes y comprobar el efecto de una corrección controlada.

**Certeza: moderada-alta** para la relación causal probable; **causa formal: no confirmada**.

### C-04 — La base primaria no aparece como causa en la evidencia disponible

**Conclusión:** la alarma crítica corresponde a `db-reporting-01`, y la comprobación de `orders-primary-01` a las 10:02 no indica saturación en ese momento.

**Certeza: alta** sobre la discrepancia de host y los valores de esa comprobación; **conclusión acotada** a la instantánea, no a todos los minutos.

### C-05 — Liveness/pérdida de pods pueden haber amplificado el impacto

**Conclusión:** los eventos comienzan tras la primera señal de espera. La arquitectura indica que liveness comparte el servidor HTTP/grupo de hilos con tráfico de negocio, mecanismo compatible con propagación del impacto.

**Certeza: moderada** como factor contribuyente plausible; la contribución cuantitativa y el mecanismo exacto no están demostrados.

### C-06 — No hay base para declarar recuperación o integridad de órdenes

**Conclusión:** la serie agregada del balanceador termina a las 10:15, pero el log de aplicación persiste hasta 10:41:30 con espera de 9000 ms y 128 pendientes. No se aportó métrica posterior ni conciliación de pedidos.

**Certeza: alta** respecto al último dato y al límite de evidencia. Recuperación e integridad completa permanecen **no verificables**.

## 6. Síntesis causal

La cadena de eventos más consistente con las observaciones disponibles es:

1. La versión 4.18.0 se publica a las 09:30 y documenta un máximo de 20

conexiones HTTP a customer-profile, frente a 200 previamente.

1. A partir de las 09:42 sube la espera de adquisición del pool.
2. A las 09:43:20 el pool registra todas sus conexiones ocupadas y una cola

pendiente.

1. La espera local crece hasta segundos; aparecen 502/503 y aumenta la latencia

de extremo a extremo.

1. Desde las 09:45 hay fallos/terminaciones de liveness y menos pods listos;

podrían amplificar la degradación, pero llegan después del inicio.

1. El HPA eleva el tamaño deseado, pero los datos no muestran que ese objetivo

se tradujera oportunamente en suficientes pods listos o resolviera la cola.

1. La alarma DB induce una línea de investigación que se corrige al identificar

que `db-reporting-01` no es la base configurada en orders-api.

1. El log de aplicación continúa degradado hasta su última muestra; no hay

prueba del final del incidente.

Esta cadena es una **explicación provisional basada en correlación temporal y telemetría**, no un experimento causal. La causa formal requiere el dato de configuración efectiva y una comparación controlada.

## 7. Evidencia necesaria para elevar la hipótesis a causa confirmada

- Historial de ConfigMap/Helm y lectura del valor real de pool en cada pod

durante el rollout y el incidente.

- Métricas completas del pool por destino/pod: conexiones adquiridas,

ocupadas/libres, cola, timeout de adquisición y duración del arriendo.

- Prueba reproducible con carga comparable en un entorno seguro, modificando

solamente la variable bajo investigación cuando sea viable.

- Correlación por pod entre pool, hilos HTTP, liveness/readiness, estado de rollout y solicitudes atendidas.
- Verificación de límites globales del downstream antes de elegir un valor de pool; aumentar conexiones no es seguro si se desconoce su capacidad.
- Telemetría posterior al último registro de aplicación para comprobar recuperación sostenida contra SLO.
- Conciliación autorizada y de solo lectura de órdenes recibidas, confirmadas, fallidas y duplicadas; no hacer cambios directos a datos.

## 8. Fuentes consultadas

- `ORIONORDERS/01_SOPORTE_INCIDENTE/02_P1-8841(2).txt`
- `ORIONORDERS/01_SOPORTE_INCIDENTE/16_chat_crisis(2).txt`
- `ORIONORDERS/02_APLICACION_DESARROLLO/05_orders-api(2).log`
- `ORIONORDERS/02_APLICACION_DESARROLLO/08_customer_profile_metrics(2).csv`
- `ORIONORDERS/02_APLICACION_DESARROLLO/13_apm_trace_breakdown(2).csv`
- `ORIONORDERS/03_INFRAESTRUCTURA/06_k8s_pod_metrics(2).csv`
- `ORIONORDERS/03_INFRAESTRUCTURA/07_k8s_events(2).log`
- `ORIONORDERS/03_INFRAESTRUCTURA/10_load_balancer_metrics(2).csv`
- `ORIONORDERS/03_INFRAESTRUCTURA/11_health_probe_config(2).yaml`
- `ORIONORDERS/04_BASE_DE_DATOS/09_db_alerts(2).txt`
- `ORIONORDERS/05_DEVOPS_CICD_RELEASE/03_RELEASE_4.18.0(2).md`
- `ORIONORDERS/05_DEVOPS_CICD_RELEASE/04_pipeline_4.18.0(3).log`
- `ORIONORDERS/00_DOCUMENTACION_GENERAL/01_ARQUITECTURA(3).md`

---